Benchmarking Sorting Algorithms

SER222

Pin-Yang Wang

1. My Hypothesis

For this experiment, I formulated hypotheses regarding the expected time complexity of insertion sort and shellsort across three distinct data sets: **Binary**, **Halves**, and **Half-Random**. Each hypothesis is based on the theoretical behavior of the algorithms relative to the data's initial order and distribution of elements.

	Insertion Sort	**Shell Sort**
Binary Data	**O(n)**, the data contains only two values and is sorted. Comparisons will immediately find the correct position.	**O(n)**, the data contains only two values and is sorted. Comparisons will immediately find the correct position.
Halves Data	**O(n)**, same as binary data, the data is sorted. Comparisons will immediately find the correct position.	**O(n)**, same as binary data, the data is sorted. Comparisons will immediately find the correct position.
Half-Random Data	**$O(n^2)$**, **t**he random half is completely unsorted. Thus, insertion sort should run in $O(n^2)$ time, because each insertion may require shifting.	**$O(n^{1 \cdot 5})$**, **t**he random half is completely unsorted. Since "h" gets smaller as h/3 in each loop, it will not be less than O(n logn). Also, it does not need to compare the elements one by one like insertion sort(when h=1, the data is almost sorted), so it will not be more than $O(n^2)$

2. Benchmark Results

```
(MLhw2) PS C:\Users\pinta\Desktop\2025asu\ser222\assignment5> java edu.ser222.m02_01.CompletedBenchmarkTool
Insertion Shellsort
Bin    0.234 0.385
Half   0.994 1.037
RanInt 0.768 2.295
```

After implementing the benchmarking tool, I obtained the following empirical b values using the doubling formula b=lg(t2/t1):

Dataset	b (Insertion sort)	b (Shell sort)
Bin	0.234	0.385
Half	0.994	1.037
RanInt	0.768	2.295

These b values represent the experimental exponent of growth.
For example, $b \approx 1$ implies near-linear growth, while $b \approx 2$ implies quadratic growth.

3. Evaluation and Validation

Comparing these results to my hypotheses:

- For Binary Data, since the dataset consists of only two kinds of values, no element shifts are required, producing sublinear growth behavior close to O(n). I assume that this is a result of timing precision limits when execution time is very short.
- For Halves Data, the dataset is partially ordered, each new value (from 0 to log n) still requires a full linear scan for comparisons, resulting in roughly linear time scaling.
- For Half-Random Data, insertion sort performed somewhat better than theoretical expectations, likely due to the relatively small input size and runtime measurement noise. On the contrary, shellsort reported a higher value. I personally assume that it is a side effect of short experimental runs or JIT warm-up bias. With a larger input size, the b value would likely shift into the theoretical range.

4. Scenario Discussion

I would choose to support keeping shellsort as the sorting algorithm.

From a theoretical standpoint, shellsort’ s gap-based insertion process makes it more robust for larger or less predictable datasets. It makes long-distance shifting possible in the early stage of the process, which provides a smoother performance curve in the worst case of the insertion sort.

Therefore, shellsort remains a safer and more scalable choice for production environments. For small arrays, insertion sort may be better, but for larger real-world workloads, shellsort’ s adaptability to random input is expected to dominate.